🏨Rooms Module – Runbook & Technical Documentation

This document describes the **Rooms Module (v1)** of the Hospitality SaaS backend. It covers purpose, architecture, API contracts, RBAC rules, business logic, and operational testing. This module is **production-ready and locked**.

Purpose & Scope

The Rooms Module is responsible for managing **hotel room inventory** within a tenant (hotel).

Key Goals

- Maintain accurate room inventory per hotel (tenant)
- Enforce role-based access (ADMIN vs STAFF)
- Ensure data integrity (no duplicate rooms per hotel)
- Support controlled room lifecycle states

Out of Scope (v1)

- Reservations / bookings
- Pricing or rate plans
- Guest assignment

Domain Model

Room Entity

Field	Type	Description
id	uuid	Primary key
roomNumber	string	Human-readable room identifier
roomType	string	DELUXE / STANDARD / etc.
capacity	number	Max occupancy
status	enum	AVAILABLE / OCCUPIED / MAINTENANCE
tenant	Tenant	Owning hotel
createdAt	timestamp	Creation time
updatedAt	timestamp	Last update time

Constraints

- **roomNumber is unique per tenant**
- Same roomNumber may exist across different tenants

Room Lifecycle (Business Rules)

Status Enum

- AVAILABLE
- OCCUPIED
- MAINTENANCE

Allowed Transitions

From	To	Allowed
AVAILABLE	OCCUPIED	✅
AVAILABLE	MAINTENANCE	✅
OCCUPIED	AVAILABLE	✅
OCCUPIED	MAINTENANCE	❌

Invalid transitions return **400 Bad Request**.

RBAC (Role-Based Access Control)

Role	List Rooms	Create Room	Update Status
ADMIN	✅	✅	✅
STAFF	✅	❌	❌

RBAC is enforced using: - `JwtAuthGuard` - `RolesGuard` - `@Roles()` decorator

API Endpoints

Create Room

```
POST /rooms
```

Role: ADMIN

Request Body

```
{
  "roomNumber": "101",
  "roomType": "DELUXE",
  "capacity": 2,
  "tenantCode": "SUNU_HOTEL"
}
```

Responses - 201 Created - 409 Conflict – duplicate room per tenant

---

List Rooms

```
GET /rooms
```

Role: ADMIN, STAFF

Response

```
[
  {
    "roomNumber": "101",
    "status": "AVAILABLE",
    "tenant": { "code": "SUNU_HOTEL" }
  }
]
```

🚡Update Room Status

```
PATCH /rooms/:id/status
```

Role: ADMIN

Request Body

```
{
  "status": "OCCUPIED"
}
```

Responses - 200 OK - 400 Bad Request (invalid transition) - 403 Forbidden (STAFF access)

Security Guarantees

- JWT-based authentication required for all endpoints
- Passwords never exposed in responses
- Tenant isolation enforced at data layer
- Duplicate data prevented at DB level

Operational Testing Runbook

Admin Login

```
export ADMIN_TOKEN=$(curl -s -X POST http://localhost:3000/auth/login
  -H "Content-Type: application/json"
  -d '{"email":"admin@sunu.com","password":"password"}' | jq -r .accessToken)
```

Staff Login

```
export STAFF_TOKEN=$(curl -s -X POST http://localhost:3000/auth/login
  -H "Content-Type: application/json"
  -d '{"email":"frontdesk@sunu.com","password":"StrongPass123"}' | jq -
r .accessToken)
```

Verify Room List (ADMIN / STAFF)

```
curl http://localhost:3000/rooms
  -H "Authorization: Bearer $ADMIN_TOKEN" | jq
```

Verify STAFF Restriction

```
curl -X POST http://localhost:3000/rooms
  -H "Authorization: Bearer $STAFF_TOKEN"
  -H "Content-Type: application/json"
  -d '{"roomNumber":"102","roomType":"STANDARD","capacity":
2,"tenantCode":"SUNU_HOTEL"}'
```

Expected: **403 Forbidden**

Stability & Freeze Notice

The following components are **LOCKED and STABLE**: - Auth - RBAC - Users - Rooms Module v1

No changes should be made without version bumping.

Next Planned Modules

- Reservations
- Room Pricing / Rate Plans
- Housekeeping
- Billing

✅**Rooms Module v1 is production-ready and verified end-to-end.**